FedMedVision: Federated Learning for Medical Imaging

Group 14: Yog Dharaskar, Nishchal Marur Nanjunda Swamy, Kriti Shahi, Karthik Ramanathan, Siddharth Pathania

Made with GAMMA

Introduction: Project Overview

Project Overview:

FedMedVision is a Federated Learning system designed to allow multiple hospitals (client nodes) to collaboratively train a deep learning model on patient datasets without transferring the data across the health institutions, ensuring data privacy and compliance with medical regulations.

Made with GAMMA

Problem Statement: Data Privacy in Healthcare

Centralized machine learning approaches pose risks related to data privacy, ownership, and regulatory compliance in healthcare. FedMedVision solves this by enabling decentralized model training while retaining patient data locally.

Made with GAMMA

Proposed Solution

FedMedVision uses a central aggregator to orchestrate federated learning across several hospitals. Each hospital trains a model locally and periodically sends model updates (not raw training data) to the central aggregator. The aggregator uses FedAvg to produce a global model.

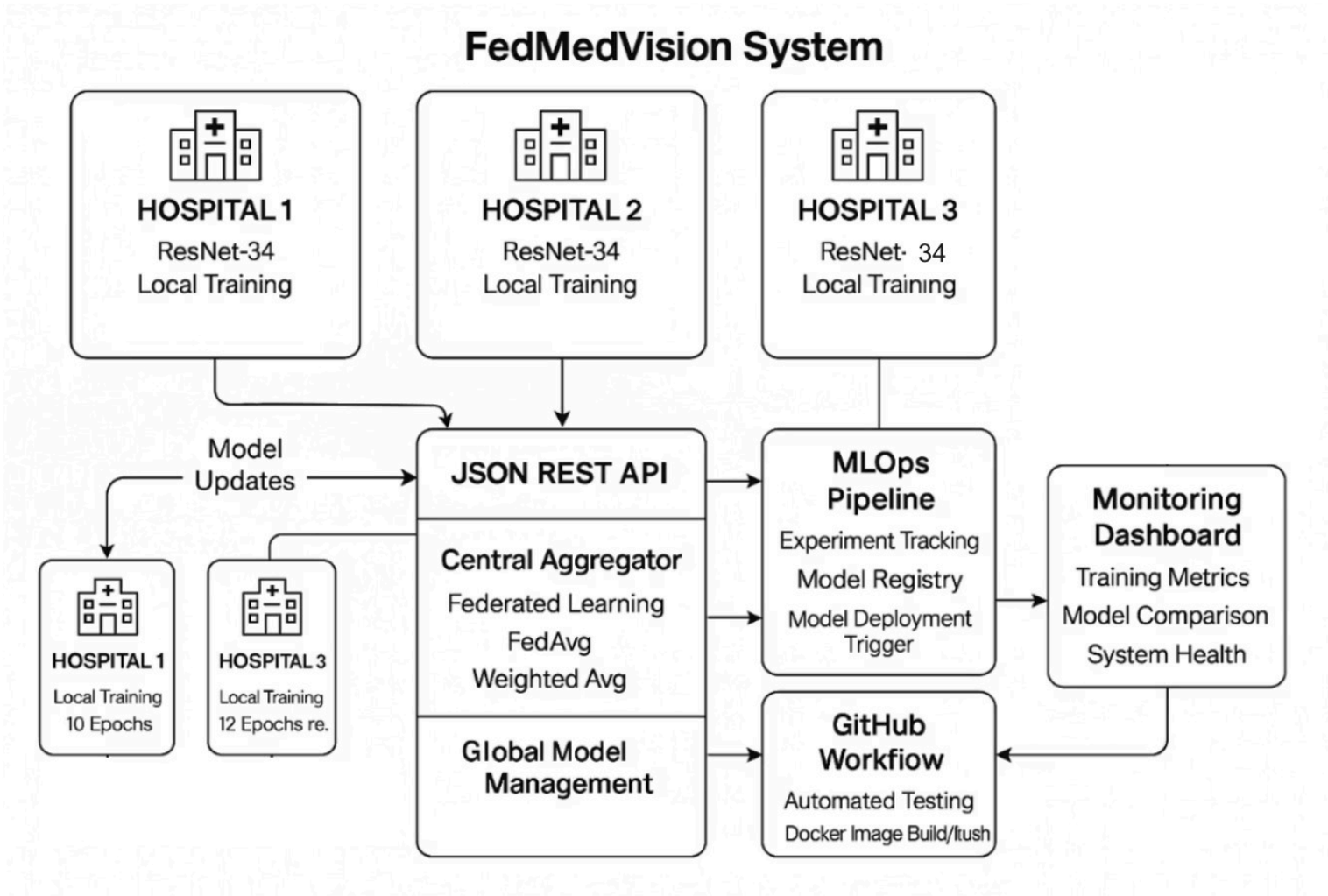
Made with GAMMA

Tools and Technologies Used

- **FastAPI**: Backend API for orchestrator and endpoints
- **SQLAlchemy + SQLite**: ORM and lightweight database
- **PyTorch**: Deep learning framework
- **MLflow**: For experiment tracking, model registry, and deployment
- **Docker**: For containerization
- **FastAI**: For plug and play model training.

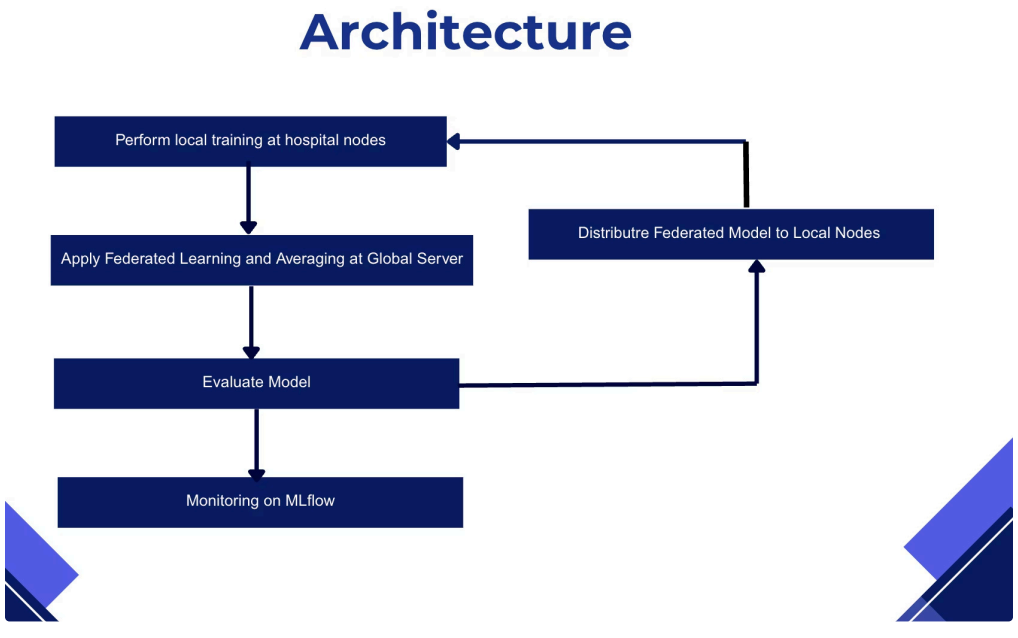
Made with GAMMA

Architecture



Made with GAMMA

Process Flow Diagram



The diagram showcases:

- Client nodes (Hospitals with local data and models)
- Central aggregator (FedAvg and model coordination)
- MLOps pipeline (MLflow)
- Monitoring dashboard (MLflow dashboard)

Made with GAMMA

Live Demo and Results

Made with GAMMA

Limitations/Future Scope

- **Security:**
  - **All API endpoints are secured with a simple API key mechanism Implement rate limiting and thorough logging for audit and debugging.**
- **Scalability:**
  - **While Docker Compose is used for the PoC, the architecture is designed to be easily migrated to Kubernetes for production scalability.**
- **Fault Tolerance:**
  - **Even though network faults are assumed negligible in the PoC, future iterations should consider adding retry logic and message queue-based asynchronous communication.**
- **Model Adaptation:**
  - **The Control API allows admins to trigger model adaptations or switch training tasks based on dashboard insights.**
- **Plug and play model support:**
  - **The model, data, and configuration can be switched to work with this federated setup for different scenarios where data privacy is crucial.**

Made with GAMMA

Thank you

Questions?

Made with GAMMA

Made with GAMMA